Laborator 7

1. Obiectivele lucrării de laborator _____ 2
2. Bune practici în dezvoltarea aplicațiilor software cu AI _____ 2
   2.1. Principii fundamentale ale dezvoltării asistate de AI _____ 2
   2.2. Verificarea, testarea și revizuirea codului generat _____ 3
3. Tema de laborator _____ 3
   3.1. Studiul documentației _____ 4
   3.2. Dezvoltarea separată a aplicațiilor MVP _____ 4
   3.3. Revizuirea codului _____ 4
4. Referințe bibliografice _____ 4



1. Obiectivele lucrării de laborator

- Evaluarea livrabilelor anterioare
- Dezvoltarea aplicației frontend (UI) pentru versiunea MVP
- Dezvoltarea aplicației backend pentru versiunea MVP
- Realizarea unei revizuiri a codului (code review)

2. Bune practici în dezvoltarea aplicațiilor software cu AI

Instrumentele bazate pe inteligență artificială (AI) pot accelera dezvoltarea software doar dacă programatorul rămâne cel care înțelege, verifică și răspunde de cod. În contextul acestui laborator, distingem două moduri în care AI poate fi utilizat în cadrul unui proiect:

- AI ca asistent de dezvoltare: asistenți de cod, agenți, chat-uri folosite pentru a scrie, explica, testa și refactoriza cod.
- AI integrat în aplicație: utilizarea unor servicii AI, de exemplu prin apelarea unui model de limbaj mare (LLM) folosind un API, similar integrării altor servicii externe.

2.1. Principii fundamentale ale dezvoltării asistate de AI

Regula de bază: în proiect ar trebui integrat doar codul care poate fi explicat la o revizie de cod. Restul principiilor decurg din această regulă.

1. **Tu ești autorul.** Codul generat de AI intră în proiect sub numele celui care îl predă, iar răspunderea pentru erori, vulnerabilități și licențe îi revine acestuia.
2. **Înțelege înainte să accepți.** Este recomandat ca fiecare modificare propusă să fie citită cu atenție. În cazul fragmentelor de cod neclare, se recomandă solicitarea unor explicații suplimentare din partea asistentului AI, iar explicația primită ar trebui confruntată cu documentația oficială.
3. **Pași mici, verificabili.** Rezultate mai bune se obțin solicitând pe rând câte o funcție, un endpoint, un test sau o funcționalitate redusă ca și complexitate, nu o aplicație întreagă dintr-un singur prompt. O modificare mică se revizuiește și se anulează ușor.
4. **Proiectarea rămâne o responsabilitate umană.** Arhitectura (niveluri, modele, controlere, rute) ar trebui stabilită de student, iar AI-ul poate fi folosit apoi pentru implementare. Un model nu cunoaște constrângerile proiectului decât dacă acestea îi sunt specificate.
5. **Controlul versiunilor e obligatoriu.** Un comit realizat înaintea fiecărei sesiuni cu un asistent sau agent AI permite revenirea oricând la o stare stabilă. Comit-urile mici, cu mesaje clare, arată ce anume a fost modificat de AI.
6. **Datele sensibile necesită protecție.** Parolele, cheile API, datele personale și alte informații confidențiale nu ar trebui transmise instrumentelor AI externe fără respectarea regulilor aplicabile privind protecția datelor.

7. **Învățarea nu se externalizează.** În context de studiu, este preferabil ca o primă variantă să fie scrisă individual și abia apoi comparată cu propunerea AI. Folosit doar ca generator, AI-ul elimină tocmai exercițiul care formează competența.

2.2. Verificarea, testarea și revizuirea codului generat

Se recomandă ca orice cod generat de AI să fie considerat o propunere ce necesită validare, similar unei contribuții din partea unui coleg nou, înainte de a fi integrat în proiect. Modelele AI pot inventa metode, pachete sau parametri care nu există și pot rezolva altă problemă decât cea cerută.

- **Compilarea și rularea imediată.** Este recomandat ca fiecare modificare să fie compilată și rulată imediat. O eroare descoperită după un pas mic costă minute; după zece pași, poate costa ore.
- **Testele înaintea încrederii.** Sunt necesare teste unitare și de integrare, inclusiv pentru cazurile-limită: id inexistent, body gol, date invalide, utilizator neautentificat.
- **Verifică testele scrise de AI.** Un test care doar repetă implementarea sau verifică un mock nu dovedește nimic. Un test valid ar trebui să pice atunci când logica din cod este greșită.
- **Testarea API-ul din exterior.** Instrumente precum Postman, cURL sau Swagger UI permit verificarea operațiilor GET, POST, PUT și DELETE cu header-e și coduri de stare reale..
- **Verificarea dependențelor.** Fiecare pachet NuGet sau npm propus trebuie confirmat ca existent, întreținut și în versiunea potrivită. Pachetele inventate de AI reprezintă un vector de atac cunoscut, deoarece numele lor pot fi înregistrate ulterior de atacatori.
- **Revizuirea modificărilor nu doar a rezultatului.** Modificările din cadrul unui comit ar trebui citite integral: un agent poate modifica fișiere care nu au fost menționate sau poate elimina validări doar pentru ca un test să treacă.
- **Preferă soluții simple.** Abstracțiile inutile, adăugate din exces de zel, este bine să fie respinse.

3. Tema de laborator



În acest laborator echipele vor preda livrabilele din laboratoarele precedente, sub forma specificată la fiecare livrabil. De asemenea, livrabilele ce vor fi implementate în acest laborator vor fi transmise în laboratorul specificat în coloana termen limită.

Observații:

- Resursele ajutătoare sunt obligatorii doar la acele livrabile la care este specificat un instrument software.
- Resursele de tip tutorial/ documentație sunt orientative, prin urmare aveți posibilitatea de a utiliza și alte tutoriale/documentații disponibile, dacă acest lucru vă ajută să implementați mai ușor și eficient respectivul livrabil.

3.1. Studiul documentației

Contribuie	Toată echipa
Responsabil	Toată echipa
Termen limită	Lab 7
Sarcini	Studiați documentația de laborator cu atenție, capitolul1. 2.

3.2. Dezvoltarea separată a aplicațiilor MVP

Contribuie	Toată Echipa
Responsabil	Team Leader
Termen limită	Lab 8
Sarcini	• Dezvoltați aplicația backend pentru versiunea MVP, urmând sarcinile planificate în Jira și practicile descrise în capitolul 2. și laboratorul 6 ○ **Observație**: În această etapă nu este necesară integrarea cu aplicația frontend, ea va fi realizată ulterior. • Dezvoltați aplicația frontend pentru versiunea MVP, urmând sarcinile planificate în Jira și practicile descrise în capitolul 2. și laboratorul 6 ○ **Observație**: În această etapă nu este necesară integrarea cu aplicația backend, ea va fi realizată ulterior.
Livrabile	• Codul dezvoltat pentru fiecare sarcină, publicat pe o ramură separată în repository-ul de pe GitHub. • Prezentarea (demonstrarea) funcționalităților dezvoltate în fața coordonatorului de laborator.
Resurse	Documentul curent, capitolul 2.

3.3. Revizuirea codului

Contribuie	Toată Echipa
Responsabil	Team Leader
Termen limită	Lab 8
Sarcini	• Cereți o recenzie (engl., code review) pentru un „pull/merge request” și adresați comentariile.
Livrabile	• Minim două comentarii primite și adresate într-un „pull/merge request” pe GitHub.
Resurse	Laboratorul 6, subcapitolul 2.5

4. Referințe bibliografice

[1] „GitHub Code Review,” [Interactiv]. Available: https://github.com/features/code-review.